

Introduction to JAX

A High-Performance Numerical Computing Library
CSE 521: Embedded Machine Learning

Omar Tahon
ID: 022025001

May 19, 2026

Outline

- ➊ Introduction to JAX
- ➋ Just-In-Time Compilation (JIT)
- ➌ Automatic Vectorization (VMAP)
- ➍ Automatic Differentiation (GRAD)
- ➎ Pytrees
- ➏ Conclusion

What is JAX?

JAX is a high-performance numerical computing library developed by Google Research. It brings together three powerful concepts:

- **NumPy API:** JAX provides a familiar `jax.numpy` interface, acting as a drop-in replacement for standard NumPy.
- **Autograd:** It allows for automatic differentiation of native Python and NumPy code.
- **XLA (Accelerated Linear Algebra):** A domain-specific compiler that accelerates math operations across CPUs, GPUs, and TPUs.

Goal: Allow researchers to write simple Python code that scales effortlessly to massive hardware accelerators.

JAX vs. Standard NumPy

While `jax.numpy` mirrors the `numpy` API, it has several superpowers under the hood.

Feature	Standard NumPy	JAX
Hardware Support	CPU only	CPU, GPU, TPU
Execution Model	Eager	Eager & Compiled (JIT)
Derivatives	Manual / Finite Diff	Automatic (<code>grad</code>)
Vectorization	Manual batching	Automatic (<code>vmap</code>)
Mutability	Mutable arrays	Immutable arrays

Table: Comparison of features between NumPy and JAX.

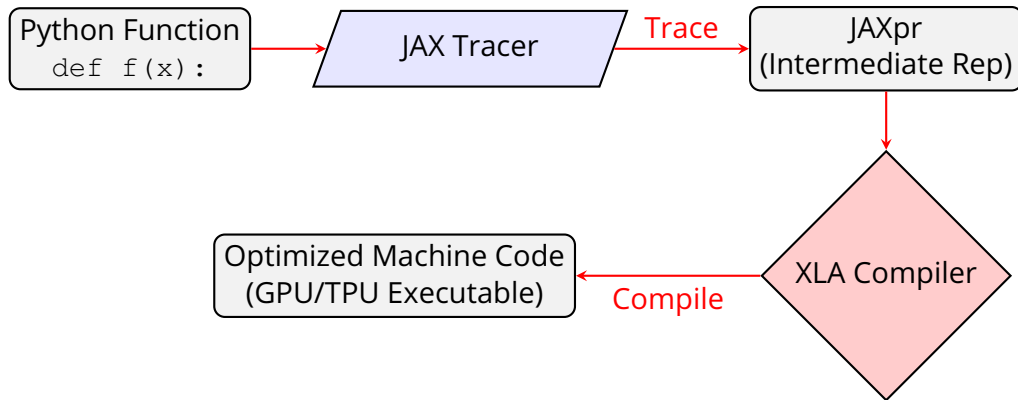
Just-In-Time Compilation (`jax.jit`)

Problem: Standard Python executes line-by-line (eagerly). This introduces significant overhead, especially for complex mathematical functions.

Solution: `jax.jit` traces your Python function and compiles it into highly optimized machine code using the **XLA** compiler.

- **Operation Fusion:** Multiple operations are merged into a single kernel, reducing memory read/write bottlenecks.
- **JAXpr Tracing:** Python logic is converted into an intermediate representation (JAXpr) before compilation.
- **Pure Functions:** Requires pure functions without side-effects (e.g., no modifying global state, no prints during execution).
- **Control Flow:** Static arguments (`static_argnums`) are required to trace dynamic Python control flow like `if/else`.

Schematic: How `jit` Works



Note: The compilation only happens once (on the first function call). Subsequent calls reuse the compiled executable instantly.

Code Example: `jax.jit` with Runtime

```
import jax.numpy as jnp
from jax import jit
import time

def selu(x, alpha=1.67, lambda=1.05):
    return lambda * jnp.where(x > 0, x, alpha * jnp.exp(x) - alpha)

fast_selu = jit(selu)
x = jnp.arange(1000000.0)

start = time.time()
selu(x) # Standard execution
print(f"Standard time: {time.time() - start:.4f} s") # ~0.0150 s

_ = fast_selu(x) # JIT warmup

start = time.time()
```

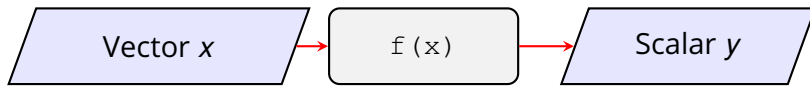
Automatic Vectorization (`jax.vmap`)

Problem: Writing code that handles varying batch sizes usually involves complex, error-prone manual loops or reshaping operations.

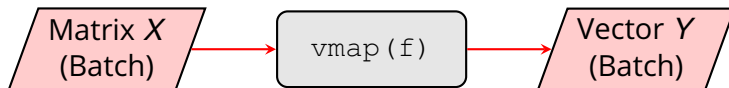
Solution: `jax.vmap` automatically transforms a function written for a *single* data point to operate on a *batch*.

- **C++/XLA Level Looping:** Pushes loop execution down to native C++ loops or SIMD instructions, avoiding Python overhead.
- **Axis Mapping:** Control input/output mapping using `in_axes` and `out_axes` to slice tensors along any dimension seamlessly.
- **Composability:** `vmap` naturally composes with `jit` to produce compiled vectorized kernels.

Schematic: How `vmap` Works



Automatic Batching



`vmap` maps the logic across a new dimension without any manual `for` loops.

Code Example: `jax.vmap` with Runtime

```
from jax import vmap
import jax.numpy as jnp
import time

def dot_product(v1, v2):
    return jnp.dot(v1, v2)

mat1, mat2 = jnp.ones((10000, 50)), jnp.ones((10000, 50))

start = time.time()
# Manual list comprehension loop
res_loop = jnp.stack([dot_product(mat1[i], mat2[i]) for i in range(10000)])
print(f"Loop time: {time.time() - start:.4f} s") # ~0.8500 s

v_dot = vmap(dot_product, in_axes=(0, 0))
start = time.time()
res_vmap = v_dot(mat1, mat2) # Vectorized execution
```

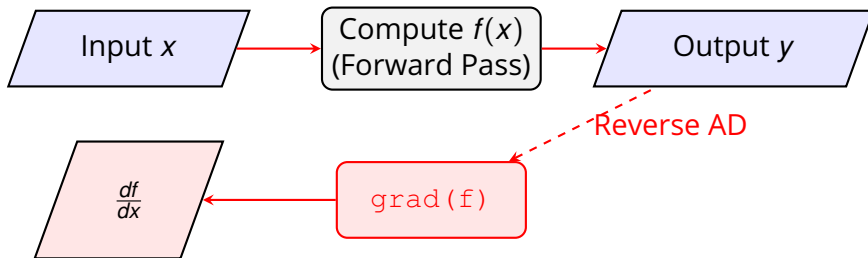
Automatic Differentiation (`jax.grad`)

Problem: Deriving gradients by hand is error-prone, and finite differences are numerically unstable and computationally expensive.

Solution: `jax.grad` computes exact, analytical gradients using **Reverse-Mode Automatic Differentiation**.

- **Reverse-Mode AD:** Highly efficient for functions with many inputs and a scalar output (e.g., loss functions), passing derivatives backward through the computational graph via the chain rule.
- **Higher-Order Derivatives:** Naturally supports chaining `grad(grad(f))` for Hessians and advanced physics-informed ML.
- **Jacobians:** Extensions like `jacfwd` and `jacrev` handle vector-valued outputs.

Schematic: Forward and Backward Pass



`grad` traces the computational graph built during the forward pass and applies the chain rule backward to compute exact derivatives.

Code Example: `jax.grad` with Runtime

```
from jax import grad
import time

#  $f(x) = (3x^2 + 2x + 1)^2$ 
def complex_loss(x):
    return (3 * x**2 + 2 * x + 1)**2

df_dx = grad(complex_loss)
x_val = 2.0

start = time.time()
for _ in range(1000):
    _ = df_dx(x_val)
print(f"GRAD 1000 calls time: {time.time() - start:.4f} s") # ~0.0350 s

# Analytical derivative validation
print("f' (x) =", df_dx(x_val)) # Output: 476.0
```

Working with Pytrees

Problem: ML parameters are stored in complex, nested lists, tuples, and dictionaries. Iterating through them during training is tedious.

Solution: JAX conceptualizes arbitrary nested Python structures as **Pytrees**.

- **Container of Leaves:** Nodes are containers (lists, dicts) and leaves are arrays/values.
- **Flattening:** `tree_flatten` and `tree_unflatten` convert nested structures into flat lists of leaves for XLA to process natively.
- **Custom Nodes:** Users can register their own Python classes as custom Pytree nodes.
- **Functional Application:** `jax.tree_map` applies a function across all leaves, bypassing complex loops.

Code Example: Pytrees with Runtime

```
from jax.tree_util import tree_map
import jax.numpy as jnp
import time

# Nested dict simulating layer parameters with arrays
params = {'layer1': jnp.ones(1000000), 'layer2': {'w': 0.5, 'b': -0.5}}

start = time.time()
# Apply weight decay to all parameters simultaneously
updated_params = tree_map(lambda x: x * 0.99, params)
print(f"Pytree update time: {time.time() - start:.4f} s") # ~0.0010 s

print(updated_params['layer2'])
# {'b': -0.495, 'w': 0.495}
```

Summary: The Power of Composable Transformations

JAX transforms the way we write mathematical code by allowing arbitrary composition of its core functions:

The Ultimate JAX Pattern

```
fast_batched_gradient_descent = jit(vmap(grad(loss_fn)))
```

- **grad**: Computes the derivative for a single example.
- **vmap**: Vectorizes the gradient computation to process an entire batch.
- **jit**: Compiles the entire batched gradient update into a single, highly optimized GPU/TPU kernel.

Thank You!

Any Questions?